Joining Tables in SQL

Data across multiple tables

Very often data will be spread across **multiple tables**. In order to perform the analyses you need to it will be important to **combine the data**. There are many different ways to combine the data and each serves a slightly different purpose.

Table A		
id	col_1	col_2
1	23-B	12
2	435	45
3	AB145	23
4	BB	56
5	435	123

Table B		
id	lookup_id	col_a
a1	1	a
a2	2	a
b1	3	b
b2	4	b
a3	5	a

Data across multiple tables

Very often data will be spread across **multiple tables**. In order to perform the analyses you need to it will be important to **combine the data**. There are many different ways to combine the data and each serves a slightly different purpose.

Table A		
id	col_1	col_2
1	23-B	12
2	435	45
3	AB145	23
4	BB	56
5	435	123

Table B		
id	lookup_id	col_a
a1	1	a
a2	2	a
b1	3	b
b2	4	b
a3	5	a

Data across multiple tables

Very often data will be spread across **multiple tables**. In order to perform the analyses you need to it will be important to **combine the data**. There are many different ways to combine the data and each serves a slightly different purpose.

Table A		
id	col_1	col_2
1	23-B	12
2	435	45
3	AB145	23
4	BB	56
5	435	123

Table B		
id	lookup_id	col_a
a1	1	a
a2	2	a
b1	3	b
b2	4	b
a3	5	a

merge_table			
id	col_1	col_2	col_a
1	23-B	12	a
2	435	45	a
3	AB145	23	b
4	BB	56	b
5	435	123	a

Keywords

- JOIN
  - INNER
  - LEFT
  - RIGHT
  - FULL
  - CROSS
  - NATURAL
  - self
  - semi
  - anti
- ON
- USING

Keywords

- JOIN
  - INNER
  - LEFT
  - RIGHT
  - FULL
  - CROSS
  - NATURAL
  - self
  - semi
  - anti
- ON
- USING

These should be familiar from Python/R.

Keywords

- JOIN
  - INNER
  - LEFT
  - RIGHT
  - FULL
  - CROSS
  - NATURAL
  - self
  - semi
  - anti
- ON
- USING

Combines every row from one table with every row from another table.

Keywords

- JOIN
  - INNER
  - LEFT
  - RIGHT
  - FULL
  - CROSS
  - NATURAL
  - self
  - semi
  - anti
- ON
- USING

Joins on all columns that are contained in both tables.

Keywords

- JOIN
  - INNER
  - LEFT
  - RIGHT
  - FULL
  - CROSS
  - NATURAL
  - self
  - semi
  - anti
- ON
- USING

Joining a table with itself.

Keywords

- JOIN
  - INNER
  - LEFT
  - RIGHT
  - FULL
  - CROSS
  - NATURAL
  - self
  - semi
  - anti
- ON
- USING

A query utilizing IN or NOT IN and a subquery.

Keywords

- JOIN
  - INNER
  - LEFT
  - RIGHT
  - FULL
  - CROSS
  - NATURAL
  - self
  - semi
  - anti
- ON
- USING

Specify the condition(s) on which to join the tables.

Keywords

- JOIN
  - INNER
  - LEFT
  - RIGHT
  - FULL
  - CROSS
  - NATURAL
  - self
  - semi
  - anti
- ON
- USING

Can be used to specify what to join on, when the tables have the same column names.

Data across multiple tables

```
SELECT a.id, col_1, col_2, col_a
FROM table_a a
INNER JOIN table_b b
ON a.id = b.lookup_id;
```

When joining tables, aliases are often used.

table_a		
id	col_1	col_2
1	23-B	12
2	435	45
3	AB145	23
4	BB	56
5	435	123

table_b		
id	lookup_id	col_a
a1	1	a
a2	2	a
b1	3	b
b2	4	b
a3	5	a

merge_table			
id	col_1	col_2	col_a
1	23-B	12	a
2	435	45	a
3	AB145	23	b
4	BB	56	b
5	435	123	a

Data across multiple tables

```
SELECT id, col_1, col_2, col_a
FROM table_a a
INNER JOIN table_b b
USING (id);
```

If the columns we are joining on have the same name, we can use USING.

table_a		
id	col_1	col_2
1	23-B	12
2	435	45
3	AB145	23
4	BB	56
5	435	123

table_b	
id	col_a
1	a
2	a
3	b
4	b
5	a

merge_table			
id	col_1	col_2	col_a
1	23-B	12	a
2	435	45	a
3	AB145	23	b
4	BB	56	b
5	435	123	a

Joining Data from Multiple Tables in the Same Query

It is possible to join more than two tables in a single query. You just need to specify what type of join and what you're joining on between each.

```
SELECT *
FROM state_crimes AS sc
INNER JOIN state
ON sc.state_id = state.id
INNER JOIN crime
ON sc.crime_id = crime.id;
```

Relational Data Tables

A database will consist of one or more **tables**.

Each table will usually hold information about one subject (eg. customers, products).

Joins can be used to connect data held in different tables.

Keys and Referential Integrity

Relationships between tables are encoded through the use of **keys**:

- **Primary key:** One or more columns which uniquely identifies each row. A table will have only one primary key.
- **Foreign key:** One or more columns in another table which refer to the primary key in another table. A table can contain more than one foreign key.

Keys and Referential Integrity

Relationships between tables are encoded through the use of **keys**:

- **Primary key:** One or more columns which uniquely identifies each row. A table will have only one primary key.
- **Foreign key:** One or more columns in another table which refer to the primary key in another table. A table can contain more than one foreign key.

Referential integrity means that when a foreign key value is used, it must reference a valid, existing primary key in the parent table. A breakdown in referential integrity can have undesirable side effects:

- Incomplete data being returned, usually with no indication of an error/"lost" records
- Strange results appearing in reports (such as products without an associated company).

Table Relationships

Tables can be related in different ways or **cardinalities.**

- **One-to-one:** Each record in one table is related to one and only one record in another. Eg. employee to payroll
- **One-to-many:** One record in the first table links to zero or more records in the second table. Eg. one customer might have multiple orders
- **Many-to-many:** Multiple records in one table can be related to each other. Eg. songs and artists; one song can have multiple artists, and one artist can have multiple songs.

Entity Relationship Diagram (ERD)

Displays the tables and how those tables relate/are connected together.

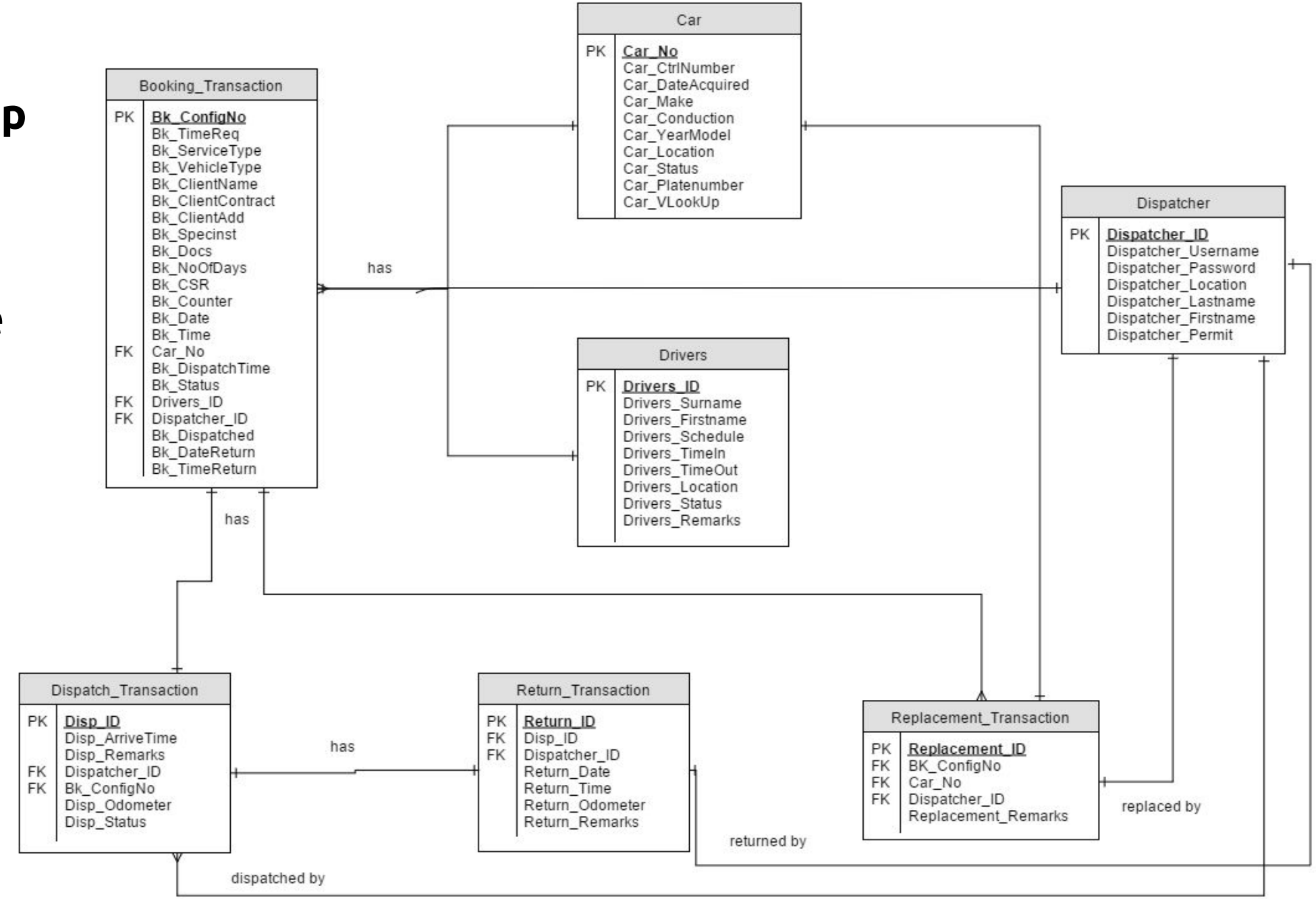

Image: https://sites.google.com/site/dracvbms/entity-relationship-diagram